
AGV-RestAPI-Doku

Release 1.1.x

Robert Berger

Mar 05, 2026

INHALT

Diese Dokumentation beschreibt den Aufbau und die Verwendung der **AGV REST API**. Sie enthält Konventionen, das allgemeine Antwortmodell (Erfolg & Fehler) und die detaillierte Beschreibung der verfügbaren Endpunkte – inklusive Parameter, Beispielanfragen und -antworten.

AGV – REST API DOKUMENTATION

1.1 Allgemeines

- **API-Version:** 1.1.x
- **Created:** Okt. 2025
- **Author:** Robert Berger
- **Protokoll:** HTTP
- **Datenformat:** Alle **Requests** (bei POST/PUT/PATCH) und **Responses** verwenden **JSON**
- **Zeichensatz:** UTF-8
- **Content-Type:** application/json
- **Fehlermodell:** Einheitliches Fehlerobjekt (siehe unten)
- **Lokalisierung:** Fehlermeldungen standardmäßig Englisch

1.2 Top-Level-Felder der API-Antworten

- `code` (*number*) – numerischer API-Statuscode (z. B. 200, 400)
- `status` (*string*) – "success" oder "error"
- `data` (*object, optional*) – Nutzlast bei **Erfolg**
- `details` (*object, optional*) – Zusatzinfos bei **Fehlern**

Genau **eines** der Felder `data` **oder** `details` ist in einer Antwort vorhanden.

1.3 Antwortmodelle

Erfolg (Success Response)

```
{
  "code": 200,
  "status": "success",
  "data": {
    "message": "pong"
  }
}
```

Fehler (Error Response)

```
{
  "code": 400,
  "status": "error",
  "details": {
    "shortDesc": "Invalid Value",
    "longDesc": "The 'channel' value is out of range."
  }
}
```

1.4 Konventionen für Requests

- **HTTP-Methoden:** GET (lesen), POST (Aktion), PUT (ersetzen)
- **Parameterübergabe:** **JSON-Body** für Nutzdaten (POST/PUT/PATCH)
- **Antwortcodes (Leitfaden):**
 - 200 OK – Erfolg mit Inhalt
 - 201 Created – Ressource erstellt
 - 204 No Content – Erfolg ohne Inhalt
 - 400 Bad Request – Ungültige Anfrage/Parameter
 - 401 Unauthorized – Authentifizierung erforderlich/fehlgeschlagen
 - 403 Forbidden – Berechtigung fehlt
 - 404 Not Found – Ressource nicht vorhanden

ENTRY-POINTS: COMMON

Note

Eine Gesamtübersicht der API findest du unter *AGV-Doku – Übersicht*.

EntryPoints:

- *GET /api/agv/common/ping*
- *POST /api/agv/common/identify*
- *POST /api/agv/common/restart*

2.1 Ping - /api/agv/common/ping

GET /api/agv/common/ping

Synopsis

Health-Check (Erreichbarkeit und Verarbeitungsfähigkeit prüfen).

Request Headers

- *Accept* – application/json

Response Headers

- *Content-Type* – application/json

Status Codes

- *200 OK* – OK

Beschreibung: Prüft, ob das System erreichbar ist und Anfragen verarbeiten kann.

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": {
    "message": "pong"
  }
}
```

Felder (``data``)

- `message (string)` – Echo-/Status-Nachricht; bei erfolgreichem Ping "pong".

2.2 Identify - /api/agv/common/identify

POST /api/agv/common/identify

Synopsis

Identifikation des Fahrzeugs auslösen.

Request Headers

- `Content-Type` – application/json

Response Headers

- `Content-Type` – application/json

JSON Parameters

- `mode (string)` – Auszuführende Aktion ("beep", "flash", "beepFlash").

Status Codes

- 200 OK – OK
- 400 Bad Request – Ungültiger Moduswert.

Beschreibung: Das Fahrzeug führt eine Identifikationssequenz aus. Es wird 5× im Sekundentakt geblinkt oder gepiepst – abhängig vom Modus.

Request Body – JSON:

```
{
  "mode": "beepFlash"
}
```

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": {
    "mode": "beepFlash"
  }
}
```

Felder (``data``)

- `mode (string)` – Der ausgeführte Modus; entspricht dem übermittelten Wert.

Response – Error (400 Bad Request):

```
{
  "code": 400,
  "status": "error",
  "details": {
    "shortDesc": "Invalid Value",
    "longDesc": "The 'mode' key must be one of the following values: 'beep', 'flash',
    ↪ 'beepFlash'."
  }
}
```

(continues on next page)

(continued from previous page)

```
}  
}
```

2.3 Restart - /api/agv/common/restart

POST /api/agv/common/restart

Synopsis

Neustart des Systems (ESP32) auslösen.

Request Headers

- **Content-Type** – application/json

Response Headers

- **Content-Type** – application/json

Status Codes

- **200 OK** – OK

Beschreibung: Führt einen Neustart des Systems aus. Beim Aufruf wird kurz rot blinkend signalisiert (2× 6 Hz), anschließend erfolgt der Neustart nach ca. 3,4 s.

Request Body – JSON: *leer*

Response – Success (200): Es wird **keine inhaltliche Rückmeldung** gesendet (kein Response-Body).

ENTRY-POINTS: BUZZER

Note

Eine Gesamtübersicht der API findest du unter *AGV-Doku – Übersicht*.

EntryPoints:

- *POST /api/agv/buzzer/off*
- *POST /api/agv/buzzer/on*
- *POST /api/agv/buzzer/beepOnce*
- *POST /api/agv/buzzer/beepInfinite*
- *POST /api/agv/buzzer/beepNTimes*

3.1 Buzzer Off - /api/agv/buzzer/off

POST /api/agv/buzzer/off

Synopsis

Schaltet den Buzzer aus.

Request Headers

- **Content-Type** – application/json

Response Headers

- **Content-Type** – application/json

Status Codes

- **200 OK** – Buzzer wurde erfolgreich ausgeschaltet.
- **400 Bad Request** – Ungültige Anfrageparameter.

Beschreibung: Schaltet den Buzzer aus und beendet alle aktiven Töne.

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": {
```

(continues on next page)

(continued from previous page)

```
"status": "off"
}
```

Felder (``data``)

- `status` (*string*) – Aktueller Zustand des Buzzers ("off").

3.2 Buzzer On - `/api/agv/buzzer/on`

POST `/api/agv/buzzer/on`

Synopsis

Schaltet den Buzzer ein.

Request Headers

- `Content-Type` – application/json

Response Headers

- `Content-Type` – application/json

JSON Parameters

- `toneFrequency_Hz` (*int*) – Tonfrequenz in Hz.

Status Codes

- 200 OK – Buzzer wurde erfolgreich eingeschaltet.
- 400 Bad Request – Ungültige Anfrageparameter.

Beschreibung: Schaltet den Buzzer ein und aktiviert einen Dauerton mit der angegebenen Frequenz.

Request Body – JSON:

```
{
  "toneFrequency_Hz": 2000
}
```

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": {
    "toneFrequency_Hz": 2000
  }
}
```

Felder (``data``):

- `toneFrequency_Hz` (*integer*) – Tatsächlich aktivierte Frequenz des Buzzers in Hertz.

Response – Error (400 Bad Request):

```
{
  "code": 400,
  "status": "error",
  "details": {
    "shortDesc": "Invalid Value",
    "longDesc": "The 'toneFrequency_Hz' value must be between 200 and 2000 Hz."
  }
}
```

3.3 Beep Once - /api/agv/buzzer/beepOnce

POST /api/agv/buzzer/beepOnce

Synopsis

Löst einen einzelnen Piepton aus.

Request Headers

- Content-Type – application/json

Response Headers

- Content-Type – application/json

JSON Parameters

- **toneFrequency_Hz** (*int*) – Tonfrequenz in Hz.
- **duration_ms** (*int*) – Tondauer in Millisekunden.

Status Codes

- 200 OK – Piepton ausgelöst.
- 400 Bad Request – Ungültige Anfrageparameter.

Beschreibung: Löst einen einzelnen Piepton mit der angegebenen Frequenz für die angegebene Dauer aus.

Request Body – JSON:

```
{
  "toneFrequency_Hz": 200,
  "duration_ms": 5000
}
```

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": {
    "toneFrequency_Hz": 200,
    "duration_ms": 5000
  }
}
```

Response – Error (400 Bad Request):

```
{
  "code": 400,
  "status": "error",
  "details": {
    "shortDesc": "Invalid Value",
    "longDesc": "The 'toneFrequency_Hz' value must be between 200 and 2000 Hz."
  }
}
```

3.4 Beep Infinite - /api/agv/buzzer/beepInfinite

POST /api/agv/buzzer/beepInfinite

Synopsis

Löst ein dauerhaftes Piepen aus (Stop per /off).

Request Headers

- Content-Type – application/json

Response Headers

- Content-Type – application/json

JSON Parameters

- **toneFrequency_Hz** (*int*) – Tonfrequenz in Hz.
- **beepFrequency_Hz** (*float*) – Piepton-Frequenz in Hz (Beep-Zyklen pro Sekunde).

Status Codes

- 200 OK – Dauerpiepen gestartet.
- 400 Bad Request – Ungültige Anfrageparameter.

Beschreibung: Startet ein dauerhaftes Piepen mit der angegebenen Frequenz, das erst durch /off beendet wird.

Request Body – JSON:

```
{
  "toneFrequency_Hz": 200,
  "beepFrequency_Hz": 0.1
}
```

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": {
    "toneFrequency_Hz": 200,
    "beepFrequency_Hz": 0.1
  }
}
```

Felder (``data``)

- **toneFrequency_Hz** (*integer*) – Tatsächlich aktivierte Frequenz des Buzzers in Hertz.

- `beepFrequency_Hz` (*float*) – Tatsächlich aktivierte Piepton-Frequenz in Hz.

3.5 Beep N-times - `/api/agv/buzzer/beepNTimes`

POST `/api/agv/buzzer/beepNTimes`

Synopsis

Löst mehrere Pieptöne hintereinander aus.

Request Headers

- `Content-Type` – `application/json`

Response Headers

- `Content-Type` – `application/json`

JSON Parameters

- `numOfBeeps` (*int*) – Anzahl der Pieptöne.
- `toneFrequency_Hz` (*int*) – Tonfrequenz in Hz.
- `beepFrequency_Hz` (*int*) – Piepton-Frequenz in Hz (Beep-Zyklen pro Sekunde).

Status Codes

- `200 OK` – Sequenz ausgelöst.
- `400 Bad Request` – Ungültige Anfrageparameter.

Beschreibung: Löst mehrere Pieptöne hintereinander aus.

Request Body – JSON:

```
{
  "toneFrequency_Hz": 200,
  "beepFrequency_Hz": 0.5,
  "numOfBeeps": 4
}
```

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": {
    "toneFrequency_Hz": 200,
    "beepFrequency_Hz": 0.5,
    "numOfBeeps": 4
  }
}
```


ENTRY-POINTS: I2C SCANNER

Note

Eine Gesamtübersicht der API findest du unter *AGV-Doku – Übersicht*.

EntryPoints:

- *GET /api/agv/i2c/scan*

4.1 I2C Device Scanner - /api/agv/i2c/scan

GET /api/agv/i2c/scan

Synopsis

Scannt den I2C-Bus und liefert eine Liste aller angeschlossenen Geräte.

Request Headers

- *Accept* – application/json

Response Headers

- *Content-Type* – application/json

Status Codes

- *200 OK* – OK

Beschreibung: Führt einen Scan des I2C-Busses durch und gibt alle erkannten Geräte zurück. Der Scanner testet die Adressen 0x01 bis 0x7E und kann bis zu 4 Geräte speichern.

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": {
    "devicesFound": 2,
    "devices": [
      {
        "index": 1,
        "address_dec": 32,
        "address_hex": "0x20",
        "name": "-"
      }
    ]
  }
}
```

(continues on next page)

(continued from previous page)

```

    },
    {
      "index": 2,
      "address_dec": 104,
      "address_hex": "0x68",
      "name": "-"
    }
  ]
}

```

Felder (`data`)

- `devicesFound` (*integer*) – Anzahl der auf dem Bus erkannten Geräte (0–4).
- `devices` (*array*) – Liste der erkannten Geräte mit folgenden Eigenschaften:
 - `index` (*integer*) – Laufende Nummer des Geräts (1-basiert).
 - `address_dec` (*integer*) – I2C-Adresse in dezimaler Notation (1–126).
 - `address_hex` (*string*) – I2C-Adresse in hexadezimaler Notation (z.B. "0x20").
 - `name` (*string*) – Name/Beschreibung des Geräts (derzeit "-"; für zukünftige Erweiterung reserviert).

Response – No Devices Found (200):

```

{
  "code": 200,
  "status": "success",
  "data": {
    "devicesFound": 0,
    "devices": []
  }
}

```

Technische Details:

- Die Scan-Funktion prüft die Präsenz von Geräten über `Wire.beginTransmission()` und `Wire.endTransmission()`.
- Maximal 4 Geräte werden in der Antwort aufgelistet (`I2C_ADDRESSBUFFERSIZE = 4`).
- Der Scan berücksichtigt nur Standard-Adressen (0x01–0x7E); reservierte Adressen (0x00, 0x7F) werden ignoriert.
- Weitere Details zur Geräte-Identifikation (z.B. Gerätetypenerkennung) sind für zukünftige Versionen geplant.

ENTRY-POINTS: INFO

Note

Eine Gesamtübersicht der API findest du unter *AGV-Doku – Übersicht*.

EntryPoints:

- *GET /api/info/agv*
- *GET /api/info/esp32/chip*
- *GET /api/info/esp32/network*
- *GET /api/info/esp32/partitions*

5.1 AGV Info – /api/info/agv

GET /api/info/agv

Synopsis

Liefert allgemeine Informationen zum AGV-System.

Request Headers

- **Accept** – application/json

Response Headers

- **Content-Type** – application/json

Status Codes

- **200 OK** – Informationen erfolgreich abgerufen.

Beschreibung: Dieser Endpunkt liefert allgemeine Systeminformationen über das AGV, darunter Betriebszeit, Name, eindeutige ID und Firmware-Details.

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": {
    "uptime_sec": 2845.399,
    "agv_name": "agv_WIUEuh",
  }
}
```

(continues on next page)

(continued from previous page)

```

    "agv_uniqueID": "unused--",
    "firmware": {
      "compile_date": "25-10-24",
      "compile_time": "16:37",
      "compile_datetime": "25-10-24 16:37",
      "version": "1.1.154"
    }
  }
}

```

Felder (`data`)

- `uptime_sec` (*float*) – Laufzeit des Systems in Sekunden.
- `agv_name` (*string*) – Aktuell konfigurierter Name des AGV.
- `agv_uniqueID` (*string*) – Eindeutige Identifikationsnummer des AGV (optional).
- **firmware** (*object*) – **Firmware-Informationen**
 - `compile_date` (*string*) – Kompilierungsdatum (TT-MM-JJ).
 - `compile_time` (*string*) – Kompilierungszeit (HH:MM).
 - `compile_datetime` (*string*) – Kombination aus Datum und Zeit.
 - `version` (*string*) – Firmware-Versionsnummer.

5.2 ESP32 Chip Info – /api/info/esp32/chip

GET /api/info/esp32/chip**Synopsis**

Liefert Systeminformationen über den ESP32-Chip.

Request Headers

- `Accept` – `application/json`

Response Headers

- `Content-Type` – `application/json`

Status Codes

- `200 OK` – Informationen erfolgreich abgerufen.

Beschreibung: Dieser Endpunkt liefert detaillierte Hardware- und Systeminformationen über den ESP32-Chip, einschließlich Speicher, CPU, Flash-Parameter, Chip-Details und des letzten Reset-Grundes. Er dient primär zu Diagnose- und Monitoring-Zwecken.

Response – Success (200):

```

{
  "code": 200,
  "status": "success",
  "data": {
    "memory": {
      "free_heap_bytes": 166548,
      "min_free_heap_bytes": 100408,

```

(continues on next page)

(continued from previous page)

```

    "max_alloc_heap_bytes": 110580,
    "total_heap": 306156
  },
  "cpu": {
    "frequency_mhz": 240,
    "core_count": 2
  },
  "flash": {
    "chip_size_bytes": 4194304,
    "chip_mode": 2,
    "chip_speed_hz": 40000000
  },
  "chip": {
    "revision": 3,
    "model": "ESP32-D0WD-V3",
    "sdk_version": "v4.4.7-dirty",
    "chip_id": "cdf9d108"
  },
  "reset": {
    "reason_id": 1,
    "reason_desc": "Power-on Reset"
  }
}

```

Felder (``data``)

- **memory (object) – Speicherinformationen**
 - free_heap_bytes (int) – Aktuell verfügbarer Heap-Speicher.
 - min_free_heap_bytes (int) – Minimaler Heap-Speicher während der Laufzeit.
 - max_alloc_heap_bytes (int) – Größter zusammenhängender Heap-Block.
 - total_heap (int) – Gesamter verfügbarer Heap.
- **cpu (object) – Prozessorinformationen**
 - frequency_mhz (int) – CPU-Frequenz in MHz.
 - core_count (int) – Anzahl der CPU-Kerne.
- **flash (object) – Flash-Speicherparameter**
 - chip_size_bytes (int) – Gesamtkapazität des Flash-Chips.
 - chip_mode (int) – Betriebsmodus des Flash (z. B. QIO/DIO).
 - chip_speed_hz (int) – Taktfrequenz des Flash-Speichers.
- **chip (object) – Chip-spezifische Details**
 - revision (int) – Hardware-Revision.
 - model (string) – Modellbezeichnung des ESP32-Chips.
 - sdk_version (string) – Version der verwendeten ESP-IDF.
 - chip_id (string) – Eindeutige Chip-ID.
- **reset (object) – Informationen zum letzten Reset**

- `reason_id (int)` – Reset-Code.
- `reason_desc (string)` – Textbeschreibung des Reset-Grundes (z. B. „Power-on Reset“).

5.3 ESP32 Network Info – `/api/info/esp32/network`

GET `/api/info/esp32/network`

Synopsis

Liefert Netzwerk- und WLAN-Informationen des ESP32.

Request Headers

- `Accept` – `application/json`

Response Headers

- `Content-Type` – `application/json`

Status Codes

- `200 OK` – Informationen erfolgreich abgerufen.

Beschreibung: Dieser Endpunkt liefert aktuelle Netzwerk- und WLAN-Parameter des ESP32, einschließlich IP-Konfiguration, MAC-Adresse, Hostname, WLAN-SSID, RSSI und Kanal.

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": {
    "network": {
      "mac_address": "08:D1:F9:CD:5F:5C",
      "ip_address": "192.168.1.132",
      "subnet_mask": "255.255.255.0",
      "ip_gateway": "192.168.1.254",
      "ip_dns": "8.8.8.8",
      "hostname": "esp32-agv",
      "ipv6": "0000:0000:0000:0000:0000:0000:0000:0000"
    },
    "wifi": {
      "rssi_dBm": -52,
      "ssid": "dd-wrt",
      "bssid": "F4:4D:5C:FB:BD:EC",
      "connection_status": 3,
      "wifi_mode": 1,
      "channel": 1
    }
  }
}
```

Felder (``data``)

- **network (object)** – Netzwerkparameter
 - `mac_address (string)` – MAC-Adresse des ESP32.
 - `ip_address (string)` – Aktuelle IPv4-Adresse.

- `subnet_mask (string)` – Subnetzmaske.
- `ip_gateway (string)` – Standard-Gateway.
- `ip_dns (string)` – DNS-Server-Adresse.
- `hostname (string)` – Geräte-Name im Netzwerk.
- `ipv6 (string)` – IPv6-Adresse (falls konfiguriert).
- **wifi (object) – WLAN-Statusinformationen**
 - `rssidBm (int)` – Empfangsfeldstärke in dBm.
 - `ssid (string)` – Aktuell verbundene WLAN-SSID.
 - `bssid (string)` – MAC-Adresse des Access Points.
 - `connection_status (int)` – WLAN-Verbindungsstatuscode.
 - `wifi_mode (int)` – WLAN-Betriebsmodus (z. B. Station, AP, etc.).
 - `channel (int)` – Aktueller WLAN-Kanal.

5.4 ESP32 Partition Info – `/api/info/esp32/partitions`

GET `/api/info/esp32/partitions`

Synopsis

Liefert Informationen über die Partitionstabelle des ESP32.

Request Headers

- `Accept` – `application/json`

Response Headers

- `Content-Type` – `application/json`

Status Codes

- **200 OK** – Informationen erfolgreich abgerufen.

Beschreibung: Dieser Endpunkt gibt eine Liste aller Partitionen des Flash-Speichers zurück, einschließlich Start- und Endadressen, Typ, Größe und Verschlüsselungsstatus.

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": {
    "partitions": [
      {
        "name": "nvs",
        "type": "data",
        "subtype": "nvs",
        "address_begin": "0x00009000",
        "address_end": "0x0000e000",
        "size_bytes": 20480,
        "size_hex_bytes": "0x00005000",
        "encrypted": "no"
      }
    ]
  }
}
```

(continues on next page)

(continued from previous page)

```

    },
    {
      "name": "otadata",
      "type": "data",
      "subtype": "ota",
      "address_begin": "0x0000e000",
      "address_end": "0x00010000",
      "size_bytes": 8192,
      "size_hex_bytes": "0x00002000",
      "encrypted": "no"
    },
    {
      "name": "app0",
      "type": "app",
      "subtype": "ota_0",
      "address_begin": "0x00010000",
      "address_end": "0x00210000",
      "size_bytes": 2097152,
      "size_hex_bytes": "0x00200000",
      "encrypted": "no"
    },
    {
      "name": "spiffs",
      "type": "data",
      "subtype": "spiffs",
      "address_begin": "0x00210000",
      "address_end": "0x003f0000",
      "size_bytes": 1966080,
      "size_hex_bytes": "0x001e0000",
      "encrypted": "no"
    },
    {
      "name": "coredump",
      "type": "data",
      "subtype": "coredump",
      "address_begin": "0x003f0000",
      "address_end": "0x00400000",
      "size_bytes": 65536,
      "size_hex_bytes": "0x00010000",
      "encrypted": "no"
    }
  ]
}

```

Felder (``data``)

- **partitions** (*array*) – Liste der Partitionseinträge. Jeder Eintrag enthält folgende Felder:
 - **name** (*string*) – Name der Partition.
 - **type** (*string*) – Typ der Partition (z. B. app oder data).
 - **subtype** (*string*) – Untertyp der Partition (z. B. nvs, spiffs).

- `address_begin` (*string*) – Startadresse im Flash.
- `address_end` (*string*) – Endadresse im Flash.
- `size_bytes` (*int*) – Größe der Partition in Bytes.
- `size_hex_bytes` (*string*) – Größe in hexadezimaler Schreibweise.
- `encrypted` (*string*) – Gibt an, ob die Partition verschlüsselt ist ("yes"/"no").

ENTRY-POINTS: STEPPER

Note

Eine Gesamtübersicht der API findest du unter *AGV-Doku – Übersicht*.

EntryPoints:

- *POST /api/agv/stepper/setVelocity*
- *POST /api/agv/stepper/enable*
- *GET /api/agv/stepper/isMoving*
- *GET /api/agv/stepper/motionMode*
- *POST /api/agv/stepper/abortMotion*
- *POST /api/agv/stepper/resetPosition*
- *GET /api/agv/stepper/getCurrentVelocity*
- *GET /api/agv/stepper/getCurrentPosition*
- *POST /api/agv/stepper/setVelocitySingle*
- *POST /api/agv/stepper/setMaxVelocityPerc*
- *POST /api/agv/stepper/setMaxAccelerationPerc*
- *POST /api/agv/stepper/setMoveAbsolute*
- *POST /api/agv/stepper/setMoveRelative*

6.1 Set Velocity (beide) – /api/agv/stepper/setVelocity

POST /api/agv/stepper/setVelocity

Synopsis

Setzt die Zielgeschwindigkeit beider Schrittmotoren in % der Maximalgeschwindigkeit.

Request Headers

- *Content-Type* – application/json

Response Headers

- *Content-Type* – application/json

JSON Parameters

- **velLeft_perc** (*float*) – Zielgeschwindigkeit linker Motor (Bereich -100.0 bis 100.0).
- **velRight_perc** (*float*) – Zielgeschwindigkeit rechter Motor (Bereich -100.0 bis 100.0).

Status Codes

- 200 OK – Geschwindigkeiten gesetzt.
- 400 Bad Request – Ungültige Werte oder fehlende Felder.

Request Body – JSON:

```
{
  "velLeft_perc": -20.0,
  "velRight_perc": 20.0
}
```

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": { "velLeft_perc": -20.0,
            "velRight_perc": 20.0
          }
}
```

Response – Error (400):

- *Invalid Value* – Werte außerhalb $[-100.0, 100.0]$ oder falsche Typen.
- *Missing Key* – Erforderliche Felder fehlen.

6.2 Enable/Disable – /api/agv/stepper/enable

POST /api/agv/stepper/enable**Synopsis**

Aktiviert oder deaktiviert die Schrittmotor-Treiber.

Request Headers

- **Content-Type** – application/json

Response Headers

- **Content-Type** – application/json

JSON Parameters

- **stepper** (*string*) – "on" oder "off".

Status Codes

- 200 OK – Zustand gesetzt.
- 400 Bad Request – Ungültiger Wert oder fehlendes Feld.

Request Body – JSON:

```
{
  "stepper": "on"
}
```

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": { "stepper": "on" }
}
```

Response – Error (400): - *Invalid Value* – stepper nicht einer der erlaubten Werte.

6.3 Is Moving – /api/agv/stepper/isMoving

GET /api/agv/stepper/isMoving

Synopsis

Prüft, ob gerade eine Bewegung aktiv ist.

Response Headers

- **Content-Type** – application/json

Status Codes

- **200 OK** – Status geliefert.

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": { "isMoving": true }
}
```

6.4 Motion Mode – /api/agv/stepper/motionMode

GET /api/agv/stepper/motionMode

Synopsis

Liefert den aktuellen Bewegungsmodus.

Response Headers

- **Content-Type** – application/json

Status Codes

- **200 OK** – Modus geliefert.

Beschreibung: Mögliche Werte: "idle", "velocity", "position", "position_planning", "mode_unknown".

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": { "motionMode": "velocity" }
}
```

6.5 Abort Motion – /api/agv/stepper/abortMotion

POST /api/agv/stepper/abortMotion

Synopsis

Bricht die aktuelle Bewegung ab.

Response Headers

- **Content-Type** – application/json

Status Codes

- **200 OK** – Abbruch initiiert.

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": { "aborted": true }
}
```

6.6 Reset Position – /api/agv/stepper/resetPosition

POST /api/agv/stepper/resetPosition

Synopsis

Setzt die aktuelle Position beider Motoren zurück.

Response Headers

- **Content-Type** – application/json

Status Codes

- **200 OK** – Position zurückgesetzt.
- **400 Bad Request** – Stepper Busy – Zurücksetzen während Bewegung nicht möglich.

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": { "positionReset": true }
}
```

Response – Error (400):

- *Stepper Busy* – Motoren bewegen sich noch.

6.7 Get Current Velocity – /api/agv/stepper/getCurrentVelocity

GET /api/agv/stepper/getCurrentVelocity

Synopsis

Liefert aktuelle Winkelgeschwindigkeit beider Motoren.

Response Headers

- Content-Type – application/json

Status Codes

- 200 OK – Geschwindigkeiten geliefert.

Beschreibung:

Die Einheit ist **rad/s**. Zusätzlich wird die konfigurierte Maximalgeschwindigkeit mitgeliefert.

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": {
    "leftVelocity": 1.2,
    "rightVelocity": -0.8,
    "unit": "radPerSec",
    "maxVelocity": 12.34
  }
}
```

6.8 Get Current Position – /api/agv/stepper/getCurrentPosition

GET /api/agv/stepper/getCurrentPosition

Synopsis

Liefert die aktuelle Position (Steps) beider Motoren.

Response Headers

- Content-Type – application/json

Status Codes

- 200 OK – Position geliefert.

Beschreibung:

Die Einheit ist **Steps**; zusätzlich wird stepsPerRevolution mitgeliefert.

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": {
    "leftPosition": 1234,
    "rightPosition": -567,
  }
}
```

(continues on next page)

(continued from previous page)

```

    "unit": "steps",
    "stepsPerRevolution": 200
  }
}

```

6.9 Set Velocity (einzeln) – /api/agv/stepper/setVelocitySingle

POST /api/agv/stepper/setVelocitySingle

Synopsis

Setzt die Zielgeschwindigkeit eines einzelnen Motors in % der Maximalgeschwindigkeit.

Request Headers

- **Content-Type** – application/json

Response Headers

- **Content-Type** – application/json

JSON Parameters

- **vel_perc** (*float*) – Zielgeschwindigkeit (Bereich -100.0 bis 100.0).
- **stepper** (*string*) – Zielmotor "left" oder "right".

Status Codes

- **200 OK** – Geschwindigkeit gesetzt.
- **400 Bad Request** – Ungültige Werte/Typen oder fehlende Felder.

Request Body – JSON:

```

{
  "vel_perc": 35.0,
  "stepper": "left"
}

```

Response – Success (200):

```

{
  "code": 200,
  "status": "success",
  "data": { "stepper": "left", "vel_perc": 35.0 }
}

```

Response – Error (400):

- *Invalid Value* – vel_perc außerhalb [-100.0, 100.0] oder stepper nicht left|right.

6.10 Set Max Velocity % – /api/agv/stepper/setMaxVelocityPerc

POST /api/agv/stepper/setMaxVelocityPerc

Synopsis

Setzt die maximale Geschwindigkeit (global) in Prozent.

Request Headers

- **Content-Type** – application/json

Response Headers

- **Content-Type** – application/json

JSON Parameters

- **maxVel_perc** (*float*) – Prozentwert (Bereich 0.0 bis 100.0).

Status Codes

- 200 **OK** – Maximalgeschwindigkeit gesetzt.
- 400 **Bad Request** – Ungültiger Wert/Typ oder Motoren bewegen sich (State).

Beschreibung: Wenn die Motoren noch laufen, kann die Maximalgeschwindigkeit nicht gesetzt werden.

Request Body – JSON:

```
{
  "maxVel_perc": 80.0
}
```

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": {
    "maxVel_perc": 80.0,
    "maxVel_radPerSec": 9.87
  }
}
```

Response – Error (400):

- *Invalid Value* – Außerhalb [0.0, 100.0] oder falscher Typ.
- *Invalid State* – Änderung während Bewegung nicht möglich.

6.11 Set Max Acceleration % – /api/agv/stepper/setMaxAccelerationPerc

POST /api/agv/stepper/setMaxAccelerationPerc

Synopsis

Setzt die maximale Beschleunigung (global) in Prozent.

Request Headers

- **Content-Type** – application/json

Response Headers

- **Content-Type** – application/json

JSON Parameters

- **maxAcc_perc** (*float*) – Prozentwert (Bereich 0.0 bis 100.0).

Status Codes

- 200 **OK** – Maximale Beschleunigung gesetzt.

- **400 Bad Request** – Ungültiger Wert/Typ oder Motoren bewegen sich (State).

Request Body – JSON:

```
{
  "maxAcc_perc": 60.0
}
```

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": {
    "maxAcc_perc": 60.0,
    "maxAcc_radPerSec2": 12.34
  }
}
```

Response – Error (400):

- *Invalid Value* – Außerhalb [0.0, 100.0] oder falscher Typ.
- *Invalid State* – Änderung während Bewegung nicht möglich.

6.12 Move Absolute – /api/agv/stepper/setMoveAbsolute

POST /api/agv/stepper/setMoveAbsolute**Synopsis**

Startet eine Positionsbewegung beider Motoren auf absolute Zielpositionen (Steps).

Request Headers

- **Content-Type** – application/json

Response Headers

- **Content-Type** – application/json

JSON Parameters

- **leftPos_steps** (*long*) – Zielposition linker Motor in Steps.
- **rightPos_steps** (*long*) – Zielposition rechter Motor in Steps.

Status Codes

- **200 OK** – Bewegung gestartet.
- **400 Bad Request** – Ungültige Werte/Typen oder Motoren beschäftigt.

Request Body – JSON:

```
{
  "leftPos_steps": 1200,
  "rightPos_steps": -800
}
```

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": {
    "leftPos_steps": 1200,
    "rightPos_steps": -800
  }
}
```

Response – Error (400):

- *Invalid Value* – Falsche Typen.
- *Invalid State* – Motoren sind beschäftigt (busy).

6.13 Move Relative – /api/agv/stepper/setMoveRelative

POST /api/agv/stepper/setMoveRelative**Synopsis**

Startet eine Relativbewegung beider Motoren (Delta in Steps).

Request Headers

- *Content-Type* – application/json

Response Headers

- *Content-Type* – application/json

JSON Parameters

- **leftDelta_steps** (*long*) – Schritt-Differenz linker Motor.
- **rightDelta_steps** (*long*) – Schritt-Differenz rechter Motor.

Status Codes

- 200 OK – Bewegung gestartet.
- 400 *Bad Request* – Ungültige Werte/Typen oder Motoren beschäftigt.

Request Body – JSON:

```
{
  "leftDelta_steps": -200,
  "rightDelta_steps": 200
}
```

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": { "leftDelta_steps": -200, "rightDelta_steps": 200 }
}
```

Response – Error (400):

- *Invalid Value* – Falsche Typen.

- *Invalid State* – Motoren sind beschäftigt (busy).

ENTRY-POINTS: PWM

Note

Eine Gesamtübersicht der API findest du unter *AGV-Doku – Übersicht*.

EntryPoints:

- *GET /api/agv/pwm/settings*
- *POST /api/agv/pwm/enable*
- *POST /api/agv/pwm/setActiveServos*
- *POST /api/agv/pwm/channel/setPosition*
- *POST /api/agv/pwm/channel/setLimits*
- *POST /api/agv/pwm/channel/setStartPosition*
- *POST /api/agv/pwm/channel/setFullSweepTime*

7.1 PWM Settings – /api/agv/pwm/settings

GET /api/agv/pwm/settings

Synopsis

Liefert den aktuellen PWM-Status, Limits und Kanal-Einstellungen.

Request Headers

- **Accept** – application/json

Response Headers

- **Content-Type** – application/json

Status Codes

- **200 OK** – Einstellungen erfolgreich abgerufen.

Beschreibung: Gibt den PWM-Gesamtstatus (aktiviert, Anzahl aktiver Servos, Maximalanzahl), Default-Werte, globale Limits sowie eine Liste der aktiven Servo-Kanäle mit deren Limits/Positionen und *fullSweepTime_ms* zurück.

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": {
    "enabled": true,
    "activeServos": 4,
    "maxServos": 8,
    "servoDefaults": {
      "min": 500,
      "max": 2500,
      "startPos": 90,
      "fullSweepTime_ms": 1000
    },
    "limits": {
      "minPos": 0,
      "maxPos": 180,
      "min_fullSweepTime_ms": 1,
      "max_fullSweepTime_ms": 5000
    },
    "servos": [
      {
        "channel": 0,
        "min": 500,
        "max": 2500,
        "startPos": 90,
        "currentPos": 90,
        "fullSweepTime_ms": 1000
      }
    ]
  }
}
```

Felder (``data``)

- `enabled` (*bool*) – PWM-Modul aktiv.
- `activeServos` (*int*) – Anzahl aktuell aktiver Servo-Kanäle.
- `maxServos` (*int*) – Maximale Kanalanzahl (Konstante `SERVO_MAX_CHANNELS`).
- `servoDefaults` (*object*) – Standardwerte für neue/Reset-Konfiguration:
 - `min` (*int*) – Standard-Minimalpuls.
 - `max` (*int*) – Standard-Maximalpuls.
 - `startPos` (*int*) – Standard-Startposition (Grad).
 - `fullSweepTime_ms` (*int*) – Standard-Zeit für 0180°-Vollausschlag.
- `limits` (*object*) – Globale Grenzen:
 - `minPos` (*int*) – Minimal erlaubte Gradposition (`SERVO_MIN_LIMIT`).
 - `maxPos` (*int*) – Maximal erlaubte Gradposition (`SERVO_MAX_LIMIT`).
 - `min_fullSweepTime_ms` (*int*) – Minimal erlaubte Vollausschlag-Zeit (ms).
 - `max_fullSweepTime_ms` (*int*) – Maximal erlaubte Vollausschlag-Zeit (ms).

- **servos (array)** – Liste aktiver Kanäle mit individuellen Werten:
 - `channel (int)` – Kanalindex.
 - `min (int), max (int)` – Kanalbezogene Pulsgrenzen.
 - `startPos (int)` – Startposition in Grad.
 - `currentPos (int)` – Aktuelle Position in Grad.
 - `fullSweepTime_ms (int)` – Vollausschlag-Zeit in ms.

7.2 PWM Enable – /api/agv/pwm/enable

POST /api/agv/pwm/enable

Synopsis

Aktiviert oder deaktiviert das PWM-Modul.

Request Headers

- **Content-Type** – application/json

Response Headers

- **Content-Type** – application/json

JSON Parameters

- **enable (bool)** – *true* zum Aktivieren, *false* zum Deaktivieren.

Status Codes

- **200 OK** – Status erfolgreich gesetzt.
- **400 Bad Request** – Ungültiger Datentyp im Request.

Beschreibung: Schaltet das PWM-Modul ein/aus. Die Antwort spiegelt den aktuellen Zustand wider.

Request Body – JSON:

```
{
  "enable": true
}
```

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": { "enabled": true }
}
```

Response – Error (400 Bad Request):

```
{
  "code": 400,
  "status": "error",
  "details": {
    "shortDesc": "Invalid Value",
    "longDesc": "The 'enable' key must be of type bool."
  }
}
```

(continues on next page)

(continued from previous page)

```
}  
}
```

7.3 Set Active Servos – /api/agv/pwm/setActiveServos

POST /api/agv/pwm/setActiveServos

Synopsis

Setzt die Anzahl der aktiven Servo-Kanäle.

Request Headers

- **Content-Type** – application/json

Response Headers

- **Content-Type** – application/json

JSON Parameters

- **activeServos** (*int*) – Anzahl aktiver Kanäle (1..`SERVO\_MAX\_CHANNELS`).

Status Codes

- **200 OK** – Anzahl erfolgreich gesetzt.
- **400 Bad Request** – Ungültiger Wert/Typ.
- **403 Forbidden** – PWM-Modul ist nicht aktiviert.

Beschreibung: Setzt die Zahl der nutzbaren Kanäle. Erfordert aktiviertes PWM-Modul.

Request Body – JSON:

```
{  
  "activeServos": 4  
}
```

Response – Success (200):

```
{  
  "code": 200,  
  "status": "success",  
  "data": { "activeServos": 4 }  
}
```

7.4 Set Channel Position – /api/agv/pwm/channel/setPosition

POST /api/agv/pwm/channel/setPosition

Synopsis

Setzt die Position eines Servo-Kanals (Grad).

Request Headers

- **Content-Type** – application/json

Response Headers

- **Content-Type** – application/json

JSON Parameters

- **channel** (*int*) – Kanalindex (0..`activeServos-1`).
- **newPosition** (*int*) – Zielposition in Grad (0..180). Wird innerhalb der Kanal-Limits geklemmt.

Status Codes

- 200 OK – Position gesetzt.
- 400 Bad Request – Ungültige Werte/Typen oder Bereichsverletzung.
- 403 Forbidden – PWM-Modul ist nicht aktiviert.

Beschreibung: Setzt die Zielposition. Liegt *newPosition* außerhalb der kanalbezogenen Limits, wird sie auf den erlaubten Bereich geklemmt. Die Antwort enthält *coerced=true*, wenn korrigiert wurde.

Request Body – JSON:

```
{
  "channel": 0,
  "newPosition": 135
}
```

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": {
    "channel": 0,
    "newPosition": 135,
    "coerced": false
  }
}
```

7.5 Set Channel Limits – /api/agv/pwm/channel/setLimits

POST /api/agv/pwm/channel/setLimits

Synopsis

Setzt Min-/Max-Grenzen eines Servo-Kanals (Grad).

Request Headers

- **Content-Type** – application/json

Response Headers

- **Content-Type** – application/json

JSON Parameters

- **channel** (*int*) – Kanalindex (0..`activeServos-1`).
- **min** (*int*) – Untere Positionsgrenze (SERVO_MIN_LIMIT..`SERVO\_MAX\_LIMIT`).
- **max** (*int*) – Obere Positionsgrenze (SERVO_MIN_LIMIT..`SERVO\_MAX\_LIMIT`, > min).

Status Codes

- 200 OK – Limits gesetzt.
- 400 Bad Request – Ungültige Werte/Bereiche.
- 403 Forbidden – PWM-Modul ist nicht aktiviert.

Beschreibung: Setzt die Positionsgrenzen des Kanals. Falls die bisherige *startPos* außerhalb der neuen Grenzen liegt, wird sie in den Bereich gezwungen. Die Antwort enthält *coerced_startPos=true*, wenn eine Korrektur erfolgte.

Request Body – JSON:

```
{
  "channel": 0,
  "min": 10,
  "max": 170
}
```

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": {
    "channel": 0,
    "min": 10,
    "max": 170,
    "startPos": 90,
    "coerced_startPos": false
  }
}
```

7.6 Set Channel Start Position – /api/agv/pwm/channel/setStartPosition

POST /api/agv/pwm/channel/setStartPosition

Synopsis

Setzt die Startposition eines Servo-Kanals (Grad).

Request Headers

- Content-Type – application/json

Response Headers

- Content-Type – application/json

JSON Parameters

- **channel** (*int*) – Kanalindex (0..`activeServos-1`).
- **startPos** (*int*) – Startposition in Grad (innerhalb der kanalbezogenen Limits).

Status Codes

- 200 OK – Startposition gesetzt.
- 400 Bad Request – Ungültige Werte/Bereiche.
- 403 Forbidden – PWM-Modul ist nicht aktiviert.

Beschreibung: Legt die Startposition fest. Der Wert muss zwischen den aktuellen Kanal-Limits liegen.

Request Body – JSON:

```
{
  "channel": 0,
  "startPos": 90
}
```

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": { "channel": 0, "startPos": 90 }
}
```

7.7 Set Channel Full Sweep Time – /api/agv/pwm/channel/setFullSweepTime

POST /api/agv/pwm/channel/setFullSweepTime

Synopsis

Setzt die Vollausschlag-Zeit (0180°) eines Servo-Kanals in Millisekunden.

Request Headers

- **Content-Type** – application/json

Response Headers

- **Content-Type** – application/json

JSON Parameters

- **channel** (*int*) – Kanalindex (0..`activeServos-1`).
- **fullSweepTime_ms** (*int*) – Vollausschlag-Zeit in ms (SERVO_MIN_FULL_SWEEP_TIME_MS..`SERVO\_MAX\_FULL\_SWEEP\_TIME\_MS`).

Status Codes

- 200 OK – Zeit gesetzt.
- 400 Bad Request – Ungültige Werte/Bereiche.
- 403 Forbidden – PWM-Modul ist nicht aktiviert.

Beschreibung: Konfiguriert, wie lange der Servo von 0° auf 180° (bzw. umgekehrt) benötigt.

Request Body – JSON:

```
{
  "channel": 0,
  "fullSweepTime_ms": 1000
}
```

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": { "channel": 0, "fullSweepTime_ms": 1000 }
}
```

ENTRY-POINTS: LED

Note

Eine Gesamtübersicht der API findest du unter *AGV-Doku – Übersicht*.

EntryPoints:

- *POST /api/agv/led/off*
- *POST /api/agv/led/on*
- *POST /api/agv/led/flashOnce*
- *POST /api/agv/led/flashInfinite*
- *POST /api/agv/led/flashNTimes*

8.1 LED Off – /api/agv/led/off

POST /api/agv/led/off

Synopsis

Schaltet die LED aus.

Request Headers

- **Accept** – application/json

Response Headers

- **Content-Type** – application/json

Status Codes

- **200 OK** – LED ausgeschaltet.

Beschreibung: Schaltet die LED sofort aus.

Request Body – JSON:

leer

Response – Success (200):

```
{  
  "code": 200,  
  "status": "success",  
}
```

(continues on next page)

(continued from previous page)

```
"data": { "status": "off" }
}
```

8.2 LED On – /api/agv/led/on

POST /api/agv/led/on

Synopsis

Schaltet die LED in einer bestimmten Farbe ein.

Request Headers

- **Content-Type** – application/json

Response Headers

- **Content-Type** – application/json

JSON Parameters

- **color** (*string*) – Eine der Farben red, green, blue, yellow, magenta, cyan, white.

Status Codes

- 200 OK – LED eingeschaltet.
- 400 Bad Request – Ungültiger Wert/Typ.

Beschreibung: Schaltet die LED dauerhaft in der angegebenen Farbe ein.

Request Body – JSON:

```
{
  "color": "red"
}
```

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": { "ledColor": "red" }
}
```

Response – Error (400 Bad Request):

```
{
  "code": 400,
  "status": "error",
  "details": {
    "shortDesc": "Invalid Value",
    "longDesc": "The 'color' value must be one of the following: 'red', 'green', 'blue',
    ↪ 'yellow', 'magenta', 'cyan', 'white'."
  }
}
```

8.3 LED Flash Once – /api/agv/led/flashOnce

POST /api/agv/led/flashOnce

Synopsis

Lässt die LED einmalig in der gewünschten Farbe für eine bestimmte Dauer leuchten.

Request Headers

- **Content-Type** – application/json

Response Headers

- **Content-Type** – application/json

JSON Parameters

- **color** (*string*) – Eine der Farben red, green, blue, yellow, magenta, cyan, white.
- **duration_ms** (*int*) – Leuchtdauer in Millisekunden (> 0).

Status Codes

- 200 OK – Flash ausgeführt.
- 400 Bad Request – Ungültige Werte/Typen.

Beschreibung: Schaltet die LED für die angegebene Zeit ein und danach wieder aus.

Request Body – JSON:

```
{
  "color": "yellow",
  "duration_ms": 750
}
```

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": { "color": "yellow", "duration_ms": 750 }
}
```

Response – Error (400 Bad Request):

```
{
  "code": 400,
  "status": "error",
  "details": {
    "shortDesc": "Invalid Value",
    "longDesc": "The 'color' value must be one of the following: 'red', 'green', 'blue',
    ↪ 'yellow', 'magenta', 'cyan', 'white'."
  }
}
```

8.4 LED Flash Infinite – /api/agv/led/flashInfinite

POST /api/agv/led/flashInfinite

Synopsis

Lässt die LED unbegrenzt mit einer bestimmten Frequenz blinken.

Request Headers

- **Content-Type** – application/json

Response Headers

- **Content-Type** – application/json

JSON Parameters

- **color** (*string*) – Eine der Farben red, green, blue, yellow, magenta, cyan, white.
- **flashFrequency_Hz** (*float*) – Blinkfrequenz in Hz (LED_MIN_FREQUENCY..`LED\_MAX\_FREQUENCY`).

Status Codes

- 200 OK – Blinksequenz gestartet.
- 400 Bad Request – Ungültige Werte/Typen.

Beschreibung: Startet ein endloses Blinken in der gewählten Farbe und Frequenz. Zum Stoppen kann z. B. /api/agv/led/off genutzt werden.

Request Body – JSON:

```
{
  "color": "cyan",
  "flashFrequency_Hz": 2.5
}
```

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": { "color": "cyan", "flashFrequency_Hz": 2.5 }
}
```

Response – Error (400 Bad Request):

```
{
  "code": 400,
  "status": "error",
  "details": {
    "shortDesc": "Invalid Value",
    "longDesc": "The 'flashFrequency_Hz' value must be between LED_MIN_FREQUENCY and LED_
    ↳MAX_FREQUENCY Hz."
  }
}
```

8.5 LED Flash N-times – /api/agv/led/flashNTimes

POST /api/agv/led/flashNTimes

Synopsis

Lässt die LED eine endliche Anzahl von Malen blinken.

Request Headers

- **Content-Type** – application/json

Response Headers

- **Content-Type** – application/json

JSON Parameters

- **flashFrequency_Hz** (*int*) – Blinkfrequenz in Hz (LED_MIN_FREQUENCY..`LED\_MAX\_FREQUENCY`).
- **numOfFlashes** (*int*) – Anzahl der Blinkvorgänge (> 0).
- **color** (*string*) – Eine der Farben red, green, blue, yellow, magenta, cyan, white.

Status Codes

- 200 OK – Blinksequenz gestartet.
- 400 Bad Request – Ungültige Werte/Typen.

Beschreibung: Blinkt die LED genau numOfFlashes-mal in der angegebenen Farbe mit der gewünschten Frequenz.

Request Body – JSON:

```
{
  "flashFrequency_Hz": 4,
  "numOfFlashes": 6,
  "color": "magenta"
}
```

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": {
    "flashFrequency_Hz": 4,
    "numOfFlashes": 6,
    "color": "magenta"
  }
}
```

Response – Error (400 Bad Request):

```
{
  "code": 400,
  "status": "error",
  "details": {
    "shortDesc": "Invalid Value",
    "longDesc": "The 'flashFrequency_Hz' value must be between LED_MIN_FREQUENCY and LED_
↪MAX_FREQUENCY Hz."
  }
}
```

(continues on next page)

(continued from previous page)

```
}  
}
```

ENTRY-POINTS: WIFI

Note

Eine Gesamtübersicht der API findest du unter *AGV-Doku – Übersicht*.

EntryPoints:

- *GET /api/agv/wifi/getWifiSettings*
- *POST /api/agv/wifi/setActiveMode*
- *POST /api/agv/wifi/setModeData*

9.1 WiFi Settings – /api/agv/wifi/getWifiSettings

GET /api/agv/wifi/getWifiSettings**Synopsis**

Liefert den aktiven WiFi-Modus und gespeicherte Zugangsdaten (AP, WiFi-01, WiFi-02).

Request Headers

- **Accept** – application/json

Response Headers

- **Content-Type** – application/json

Status Codes

- **200 OK** – Einstellungen erfolgreich abgerufen.

Beschreibung: Gibt den aktuell gesetzten WiFi-Modus sowie die aus dem NVS gespeicherten SSIDs und Passwörter für die Profile **AP**, **wifi-01** und **wifi-02** zurück.

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": {
    "wifiMode": "wifi-01",
    "ap": { "ssid": "agv-ap", "passwd": "ap-password" },
    "wifi_01": { "ssid": "home-wifi", "passwd": "secret-123" },
    "wifi_02": { "ssid": "lab-wifi", "passwd": "lab-secret" }
  }
}
```

(continues on next page)

(continued from previous page)

```
}
}
```

Felder (`data`)

- **wifiMode** (*string*) – Aktiver Modus: "ap", "wifi-01" oder "wifi-02".
- **ap** (*object*) – Zugangsdaten für Access-Point-Modus. - **ssid** (*string*), **passwd** (*string*)
- **wifi_01** (*object*) – Zugangsdaten Profil 1. - **ssid** (*string*), **passwd** (*string*)
- **wifi_02** (*object*) – Zugangsdaten Profil 2. - **ssid** (*string*), **passwd** (*string*)

9.2 Set Active WiFi Mode – /api/agv/wifi/setActiveMode

POST /api/agv/wifi/setActiveMode**Synopsis**

Setzt den aktiven WiFi-Modus.

Request Headers

- **Content-Type** – application/json

Response Headers

- **Content-Type** – application/json

JSON Parameters

- **newMode** (*string*) – Neuer Modus, einer von "ap", "wifi-01", "wifi-02".

Status Codes

- **200 OK** – Modus erfolgreich gesetzt.
- **400 Bad Request** – Ungültiger Wert/Typ.
- **403 Forbidden** – Zugriff verweigert.

Beschreibung: Aktiviert den angegebenen WiFi-Modus. Die Antwort spiegelt den tatsächlich gesetzten Modus wider.**Request Body – JSON:**

```
{
  "newMode": "wifi-02"
}
```

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": { "newMode": "wifi-02" }
}
```

Response – Error (400 Bad Request):

```
{
  "code": 400,
  "status": "error",
  "details": {
    "shortDesc": "Bad Request",
    "longDesc": "The key 'newMode' can only have the values 'ap', 'wifi-01' or 'wifi-02'."
  }
}
```

9.3 Set WiFi Mode Data – /api/agv/wifi/setModeData

POST /api/agv/wifi/setModeData

Synopsis

Hinterlegt SSID und Passwort für ein WiFi-Profil (AP, wifi-01 oder wifi-02).

Request Headers

- **Content-Type** – application/json

Response Headers

- **Content-Type** – application/json

JSON Parameters

- **mode** (*string*) – Zielprofil: "ap", "wifi-01", "wifi-02".
- **ssid** (*string*) – SSID (Länge 1..32 Zeichen).
- **passwd** (*string*) – Passwort (Länge 8..63 Zeichen).

Status Codes

- 200 OK – Zugangsdaten erfolgreich gespeichert.
- 400 Bad Request – Ungültige Werte/Typen.
- 403 Forbidden – Zugriff verweigert.

Beschreibung: Speichert SSID und Passwort für das gewählte Profil im NVS. Die Antwort enthält die soeben gesetzten Werte.

Request Body – JSON:

```
{
  "mode": "wifi-01",
  "ssid": "home-wifi",
  "passwd": "secret-123"
}
```

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": {
    "mode": "wifi-01",
```

(continues on next page)

(continued from previous page)

```
"ssid": "home-wifi",  
"passwd": "secret-123"  
}  
}
```

Response – Error (400 Bad Request):

```
{  
  "code": 400,  
  "status": "error",  
  "details": {  
    "shortDesc": "Bad Request",  
    "longDesc": "The key 'mode' can only have the values 'ap', 'wifi-01' or 'wifi-02'."  
  }  
}
```

ENTRY-POINTS: WHEREISMYAGV

Note

Eine Gesamtübersicht der API findest du unter *AGV-Doku – Übersicht*.

EntryPoints:

- *GET /api/agv/whereIsMyAGV/status*
- *POST /api/agv/whereIsMyAGV/enable*
- *POST /api/agv/whereIsMyAGV/settings*

10.1 WhereIsMyAGV Status – /api/agv/whereIsMyAGV/status

GET /api/agv/whereIsMyAGV/status

Synopsis

Liefert den aktuellen Status der “WhereIsMyAGV”-Funktion.

Request Headers

- **Accept** – application/json

Response Headers

- **Content-Type** – application/json

Status Codes

- **200 OK** – Status erfolgreich abgerufen.
- **403 Forbidden** – Zugriff verweigert.

Beschreibung: Zeigt an, ob die „WhereIsMyAGV“-Funktion aktiviert ist, und gibt die aktuell gespeicherte Gruppenzugehörigkeit sowie das Secret-Token zurück.

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": {
    "enable": true,
    "group": "AGV-TEAM-01",
```

(continues on next page)

(continued from previous page)

```
"secret": "mySecretToken123"
}
```

Felder (``data``):

- `enable` (*bool*) – Gibt an, ob die Funktion aktiviert ist.
- `group` (*string*) – Gruppenname, mindestens 3 Zeichen.
- `secret` (*string*) – Geheimes Token zur Identifikation.

10.2 WhereIsMyAGV Enable – `/api/agv/whereIsMyAGV/enable`

POST `/api/agv/whereIsMyAGV/enable`

Synopsis

Aktiviert oder deaktiviert die „WhereIsMyAGV“-Funktion.

Request Headers

- `Content-Type` – `application/json`

Response Headers

- `Content-Type` – `application/json`

JSON Parameters

- `enable` (*bool*) – `true` zum Aktivieren, `false` zum Deaktivieren.

Status Codes

- `200 OK` – Status geändert.
- `400 Bad Request` – Ungültiger Wert/Typ.
- `403 Forbidden` – Zugriff verweigert.

Beschreibung: Schaltet die „WhereIsMyAGV“-Funktion ein oder aus.

Request Body – JSON:

```
{ "enable": true }
```

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": { "enable": true }
}
```

Response – Error (400 Bad Request):

```
{
  "code": 400,
  "status": "error",
  "details": {
```

(continues on next page)

(continued from previous page)

```

    "shortDesc": "Invalid Value",
    "longDesc": "The 'enable' key must be of type bool."
  }
}

```

10.3 WhereIsMyAGV Settings – /api/agv/whereIsMyAGV/settings

POST /api/agv/whereIsMyAGV/settings

Synopsis

Legt die Gruppen- und Secret-Daten für die „WhereIsMyAGV“-Funktion fest.

Request Headers

- **Content-Type** – application/json

Response Headers

- **Content-Type** – application/json

JSON Parameters

- **group** (*string*) – Gruppenname (mind. 3 Zeichen).
- **secret** (*string*) – Geheimschlüssel zur Authentifizierung.

Status Codes

- **200 OK** – Einstellungen erfolgreich übernommen.
- **400 Bad Request** – Ungültige Werte/Typen.
- **403 Forbidden** – Zugriff verweigert.

Beschreibung: Aktualisiert die Gruppen-ID und das Secret für den „WhereIsMyAGV“-Service. Ungültige Werte (z. B. zu kurze Gruppennamen) führen zu einem 400-Fehler.

Request Body – JSON:

```

{
  "group": "AGV-TEAM-01",
  "secret": "mySecretToken123"
}

```

Response – Success (200):

```

{
  "code": 200,
  "status": "success",
  "data": {
    "group": "AGV-TEAM-01",
    "secret": "mySecretToken123"
  }
}

```

Response – Error (400 Bad Request):

```
{
  "code": 400,
  "status": "error",
  "details": {
    "shortDesc": "Invalid Value",
    "longDesc": "The 'group' value must be at least 3 characters long."
  }
}
```

ENTRY-POINTS: NVS (NON-VOLATILE STORAGE)

Note

Eine Gesamtübersicht der API findest du unter *AGV-Doku – Übersicht*.

EntryPoints:

- *GET /api/nve/namespaces*
- *POST /api/nve/listKeysAndValues*

11.1 List NVS Namespaces – /api/nve/namespaces

GET /api/nve/namespaces

Synopsis

Gibt alle in der NVS-Partition vorhandenen Namespaces zurück.

Request Headers

- *Accept* – application/json

Response Headers

- *Content-Type* – application/json

Status Codes

- *200 OK* – Erfolgreich.
- *403 Forbidden* – Zugriff verweigert.

Beschreibung: Liest die NVS-Partition des ESP32 aus und listet alle vorhandenen Namespaces (z. B. Module, Konfigurationsbereiche) auf.

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": {
    "namespaces": [
      "wifi",
      "system",
      "agv_settings",
```

(continues on next page)

(continued from previous page)

```

    "sensors"
  ]
}
}

```

Felder (`data`):

- **namespaces** (*array of string*) – Liste aller vorhandenen Namespace-Namen in der NVS-Partition.

11.2 List Keys and Values – /api/nve/listKeysAndValues

POST /api/nve/listKeysAndValues

Synopsis

Listet alle Schlüssel und deren Werte innerhalb eines gegebenen NVS-Namespace.

Request Headers

- **Content-Type** – application/json

Response Headers

- **Content-Type** – application/json

JSON Parameters

- **namespace** (*string*) – Name des auszulesenden Namespaces.

Status Codes

- **200 OK** – Erfolgreich.
- **400 Bad Request** – Ungültiger Wert/Typ.
- **403 Forbidden** – Zugriff verweigert.
- **500 Internal Server Error** – Interner Fehler beim Zugriff auf NVS.

Beschreibung: Öffnet den angegebenen NVS-Namespace und liest alle enthaltenen Schlüssel, Datentypen und Werte aus. Unterstützt werden alle NVS-Typen (z. B. int32, string, blob).

Request Body – JSON:

```

{
  "namespace": "agv"
}

```

Response – Success (200):

```

{
  "code": 200,
  "status": "success",
  "data": {
    "namespace": "wifi",
    "items": [
      { "key": "ssid", "typeId": 33, "typeName": "string", "value": "MyWiFi" },
      { "key": "pw", "typeId": 33, "typeName": "string", "value": "12345678" },
      { "key": "channel", "typeId": 4, "typeName": "int32", "value": 6 }
    ]
  }
}

```

(continues on next page)

(continued from previous page)

```
}
}
```

Felder (`data`):

- **namespace** (*string*) – Der angefragte Namespace.
- **items** (*array of object*) – Schlüssel-Wert-Paare des Namespaces.
 - **key** (*string*) – Schlüsselname
 - **typeId** (*int*) – NVS-interne Typ-ID
 - **typeName** (*string*) – Menschlich lesbarer Typ (z. B. `int32`, `string`, `blob`)
 - **value** (*mixed*) – Wert (je nach Typ Zahl, String oder Byte-Array)

Response – Error (400 Bad Request):

```
{
  "code": 400,
  "status": "error",
  "details": {
    "shortDesc": "Invalid Value",
    "longDesc": "The 'namespace' key must be of type string."
  }
}
```

Response – Error (500 Internal Server Error):

```
{
  "code": 500,
  "status": "error",
  "details": {
    "shortDesc": "NVS Error",
    "longDesc": "Failed to open NVS namespace: wifi"
  }
}
```


ENTRY-POINTS: LINE FOLLOWER

Note

Eine Gesamtübersicht der API findest du unter *AGV-Doku – Übersicht*.

EntryPoints:

- *GET /api/agv/linefollower/settings*
- *GET /api/agv/linefollower/sensors*
- *POST /api/agv/linefollower/enable*
- *POST /api/agv/linefollower/setSampleDeltaTime*

12.1 Line Follower Sensors – /api/agv/linefollower/settings

GET /api/agv/linefollower/settings

Synopsis

Liefert die aktuellen Einstellungen des Line Follower-Moduls.

Request Headers

- *Accept* – application/json

Response Headers

- *Content-Type* – application/json

Status Codes

- *200 OK* – Einstellungen erfolgreich abgerufen.

Beschreibung: Liefert die aktuellen Einstellungen des Line Follower-Moduls.

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": {
    "enabled": false,
    "sampleDeltaTime": 200
  }
}
```

Felder (``data``)

- `enabled` (*bool*) – Line Follower-Modul aktiv/inaktiv.
- `sampleDeltaTime` (*int*) – Delta Time in Millisekunden.

12.2 Line Follower Sensors – `/api/agv/linefollower/sensors`

GET `/api/agv/linefollower/sensors`**Synopsis**

Liefert die Sensorwerte des Line Followers.

Request Headers

- `Accept` – `application/json`

Response Headers

- `Content-Type` – `application/json; charset=utf-8`

Status Codes

- `200 OK` – Sensorwerte.

Beschreibung: Liefert für jeden der Sensoren des Line Follower den zuletzt ausgelesenen Wert. Die Sensorwerte des Line Followers werden intern alle 10ms aktualisiert.

Request Body – JSON:

leer

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": {
    "timestamp_ms": 55424,
    "sensors": [2, 16, 202, 72, 7]
  }
}
```

Felder (``data``):

- `timestamp_ms` (*unsigned long*) - Relativer Zeitpunkt des Auslesens der Sensorwerte (in Millisekunden).
- `sensors` (*array of integers*) – Liste aller Sensorwerte (Array Index 0–4 entsprechen Sensor 1–5).

12.3 Line Follower Sensors – `/api/agv/linefollower/enable`

POST `/api/agv/linefollower/enable`**Synopsis**

Aktiviert oder deaktiviert das Line Follower-Modul.

Request Headers

- `Accept` – `application/json`

Response Headers

- **Content-Type** – application/json; charset=utf-8

Status Codes

- **200 OK** – Status erfolgreich gesetzt.
- **400 Bad Request** – Ungültiger Datentyp oder Wert im Request.

Beschreibung: Schaltet das Line Follower-Modul ein/aus. Die Antwort spiegelt den aktuellen Zustand wider.

```
{
  "enable": true
}
```

Response – Success (200):

```
{
  "code": 200,
  "status": "success",
  "data": { "enabled": true }
}
```

Response – Error (400 Bad Request):

```
{
  "code": 400,
  "status": "error",
  "details": {
    "shortDesc": "Invalid Value",
    "longDesc": "The 'enable' key must be of type bool."
  }
}
```

12.4 Set Sample Delta Time – /api/agv/linefollower/setSampleDeltaTime

POST /api/agv/linefollower/setSampleDeltaTime

Synopsis

Setzt das Zeit zwischen zwei Anfragen der Line Follower Daten.

Request Headers

- **Content-Type** – application/json

Response Headers

- **Content-Type** – application/json

JSON Parameters

- **sampleDeltaTime** (*unsigned int*) – Delta Time in Millisekunden.

Status Codes

- **200 OK** – Wert erfolgreich gesetzt.
- **400 Bad Request** – Ungültiger Datentyp oder Wert im Request.

Beschreibung: Legt fest, in welchem zeitlichen Abstand die Firmware die Werte des Line-Follower-Sensors per Polling abfragt. Aus diesem Intervall ergibt sich die Frequenz, mit der die Sensordaten aktualisiert werden.

Request Body – JSON:

```
{  
  "sampleDeltaTime": 100  
}
```

Response – Success (200):

```
{  
  "code": 200,  
  "status": "success",  
  "data": {  
    "sampleDeltaTime": 100  
  }  
}
```

Response – Error (400 Bad Request):

```
{  
  "code": 400,  
  "status": "error",  
  "details": {  
    "shortDesc": "Invalid Value",  
    "longDesc": "The 'sampleDeltaTime' key must be of type unsigned integer."  
  }  
}
```

HTTP ROUTING TABLE

/api

GET /api/agv/common/ping, ??	POST /api/agv/stepper/resetPosition, ??
GET /api/agv/i2c/scan, ??	POST /api/agv/stepper/setMaxAccelerationPerc, ??
GET /api/agv/linefollower/sensors, ??	POST /api/agv/stepper/setMaxVelocityPerc, ??
GET /api/agv/linefollower/settings, ??	POST /api/agv/stepper/setMoveAbsolute, ??
GET /api/agv/pwm/settings, ??	POST /api/agv/stepper/setMoveRelative, ??
GET /api/agv/stepper/getCurrentPosition, ??	POST /api/agv/stepper/setVelocity, ??
GET /api/agv/stepper/getCurrentVelocity, ??	POST /api/agv/stepper/setVelocitySingle, ??
GET /api/agv/stepper/isMoving, ??	POST /api/agv/whereIsMyAGV/enable, ??
GET /api/agv/stepper/motionMode, ??	POST /api/agv/whereIsMyAGV/settings, ??
GET /api/agv/whereIsMyAGV/status, ??	POST /api/agv/wifi/setActiveMode, ??
GET /api/agv/wifi/getWifiSettings, ??	POST /api/agv/wifi/setModeData, ??
GET /api/info/agv, ??	POST /api/nve/listKeysAndValues, ??
GET /api/info/esp32/chip, ??	
GET /api/info/esp32/network, ??	
GET /api/info/esp32/partitions, ??	
GET /api/nve/namespaces, ??	
POST /api/agv/buzzer/beepInfinite, ??	
POST /api/agv/buzzer/beepNTimes, ??	
POST /api/agv/buzzer/beepOnce, ??	
POST /api/agv/buzzer/off, ??	
POST /api/agv/buzzer/on, ??	
POST /api/agv/common/identify, ??	
POST /api/agv/common/restart, ??	
POST /api/agv/led/flashInfinite, ??	
POST /api/agv/led/flashNTimes, ??	
POST /api/agv/led/flashOnce, ??	
POST /api/agv/led/off, ??	
POST /api/agv/led/on, ??	
POST /api/agv/linefollower/enable, ??	
POST /api/agv/linefollower/setSampleDeltaTime, ??	
POST /api/agv/pwm/channel/setFullSweepTime, ??	
POST /api/agv/pwm/channel/setLimits, ??	
POST /api/agv/pwm/channel/setPosition, ??	
POST /api/agv/pwm/channel/setStartPosition, ??	
POST /api/agv/pwm/enable, ??	
POST /api/agv/pwm/setActiveServos, ??	
POST /api/agv/stepper/abortMotion, ??	
POST /api/agv/stepper/enable, ??	